Relatório de Testes do Sistema AnotAI

Gabriel Tomita

Instituto Federal do Paraná, Campus Paranavaí

Bacharelado em Engenharia de Software

Disciplina de Teste de Software

3 de novembro de 2026

Resumo

Este relatório apresenta a avaliação do módulo financeiro do AnotAI por quatro perspectivas complementares: teste de caixa-preta, teste de caixa-branca, análise de acessibilidade e teste de carga. A execução funcional registrada no Testiny concluiu 10 de 10 casos com resultado aprovado, apoiada por uma suíte Django de 29 testes concluída sem falhas. A análise estrutural mediu 87,1% de cobertura de instruções, 63,8% de cobertura de decisões e 84,2% de cobertura combinada. A auditoria automática de seis telas segundo regras WCAG 2.2 de níveis A e AA encontrou uma regra violada em três elementos da lista de parcelas. O ensaio de carga local processou 731 requisições sem erros, mas a latência p95 aumentou de 0,98 s com 10 usuários virtuais para 5,56 s com 50 usuários, enquanto a vazão permaneceu próxima de 15 a 16 requisições por segundo. Os resultados confirmam o comportamento funcional previsto e indicam prioridades objetivas para ampliar a cobertura de decisões, corrigir nomes acessíveis e investigar o limite de desempenho em ambiente semelhante ao de produção.

Palavras-chave: teste de software; caixa-preta; caixa-branca; acessibilidade; teste de carga; Django; Testiny.

Abstract

This report evaluates the financial module of AnotAI from four complementary perspectives: black-box testing, white-box testing, accessibility analysis, and load testing. The functional execution recorded in Testiny completed all 10 test cases with a passed result, supported by a Django suite of 29 tests with no failures. Structural analysis measured 87.1% statement coverage, 63.8% branch coverage, and 84.2% combined coverage. An automated audit of six screens against WCAG 2.2 Level A and AA rules found one violated rule affecting three elements on the installment list. The local load test completed 731 requests without errors, but p95 latency increased from 0.98 s with 10 virtual users to 5.56 s with 50 users, while throughput remained close to 15 to 16 requests per second. The results confirm the expected functional behavior and identify priorities for expanding branch coverage, correcting accessible names, and evaluating performance in a production-like environment.

Keywords: software testing; black-box testing; white-box testing; accessibility; load testing; Django; Testiny.

1 Introdução

O objeto deste trabalho é o módulo financeiro do AnotAI, aplicação Web desenvolvida em Django no projeto da disciplina de Novas Aplicações em Engenharia de Software. O módulo permite cadastrar pessoas, categorias, centros, formas de pagamento, lançamentos e parcelas, além de apresentar indicadores financeiros e separar os dados por usuário.

O planejamento anterior foi revisto conforme o tutorial de criação de Plano de Testes no Testiny. Foram confirmados o objetivo, o objeto de teste, o escopo, a estratégia, as técnicas, o ambiente, os critérios de aprovação, o cronograma, os riscos e os casos previstos. Também foram verificados os quatro artefatos relacionados no tutorial: projeto, plano, casos de teste e execução. O Testiny contém o projeto **AnotAI - Gestão Financeira**, o plano **Plano de Testes - AnotAI Gestão Financeira**, 10 casos funcionais e a execução fechada **TR-1 - Execução Funcional AnotAI - 03/11/2026**.

Este relatório amplia o trabalho funcional inicial para atender à etapa seguinte da atividade. O texto descreve o procedimento, os dados, os resultados e as limitações de cada modalidade de teste. A avaliação foi realizada sobre o commit `7bb341164df32499b8c71d0ef6cfa421f1aeed28` do repositório `rafaelzottesso/naes2026`, em 3 de novembro de 2026.

2 Objeto e ambiente de teste

O escopo principal compreendeu regras de negócio e telas do aplicativo `financeiro`. As verificações abrangeram validação de pessoas, geração de parcelas, consistência de pagamentos, classificação de situação, isolamento por usuário, navegação, semântica de interface e comportamento sob consultas simultâneas.

O ambiente foi isolado do banco publicado. Os testes funcionais automatizados usaram SQLite em memória, e os ensaios de acessibilidade e carga utilizaram uma base SQLite local criada exclusivamente para esta avaliação. A aplicação foi executada com Python 3.12.14, Django 5.2.14 e servidor de desenvolvimento em `127.0.0.1`. A análise de acessibilidade usou Google Chrome 156.0.8078.74 e axe-core 4.14.0. Nenhum teste de carga foi dirigido ao serviço em produção.

3 Metodologia

O teste de caixa-preta derivou casos das entradas, saídas e regras observáveis, sem usar a estrutura interna como critério de aprovação. O teste de caixa-branca examinou a execução das instruções e decisões do código com coverage.py. A análise de acessibilidade combinou regras automáticas WCAG 2.2 A e AA com inspeções de estrutura e uma amostra de navegação por teclado. O teste de carga aplicou três estágios de concorrência sobre quatro rotas autenticadas do módulo financeiro.

Um caso funcional foi aprovado quando o resultado observado correspondeu ao resultado esperado. A cobertura estrutural foi registrada como medida de alcance, sem ser tratada isoladamente como prova de ausência de defeitos. Na acessibilidade, uma tela sem violações automáticas não foi declarada plenamente acessível, pois critérios dependentes de julgamento humano permanecem fora do alcance da automação. No ensaio de carga, sucesso significou resposta HTTP 200 sem redirecionamento indevido para autenticação; a interpretação também considerou latência e vazão.

4 Relato do teste de caixa-preta

4.1 Objetivo e técnica

O teste de caixa-preta verificou o comportamento funcional do módulo financeiro a partir das entradas e saídas esperadas. Foram usadas classes de equivalência para dados válidos e inválidos, análise de valores de fronteira em quantidades e valores monetários, tabela de decisão para pagamento e transição de estados para parcelas pagas, vencidas e em aberto.

Os casos foram especificados no Testiny antes do registro dos resultados. A execução foi apoiada pela suíte automatizada do Django, complementada por uma validação específica do documento de pessoa. A suíte encontrou 29 testes e concluiu todos com sucesso. No Testiny, os 10 casos foram marcados como aprovados e a execução foi encerrada com 100% de conclusão.

4.2 Casos e resultados

ID	Verificação	Técnica principal	Resultado
CT01	Cadastrar pessoa com CPF válido	Classe válida	Aprovado
CT02	Rejeitar CPF ou CNPJ com quantidade inválida de dígitos	Classe inválida e fronteira	Aprovado
CT03	Impedir documento duplicado para o mesmo usuário	Regra de unicidade	Aprovado
CT04	Cadastrar lançamento à vista	Classe válida	Aprovado
CT05	Dividir R$ 100,00 em três parcelas e ajustar o resto	Fronteira e cálculo monetário	Aprovado
CT06	Rejeitar parcelamento sem intervalo	Classe inválida	Aprovado
CT07	Rejeitar valor líquido menor ou igual a zero	Valor limite	Aprovado
CT08	Exigir valor e data de pagamento em conjunto	Tabela de decisão	Aprovado
CT09	Classificar parcelas como paga, vencida e em aberto	Transição de estados	Aprovado
CT10	Isolar dados financeiros por usuário	Controle de acesso	Aprovado

O CT02 foi executado também por um roteiro independente, que forneceu um documento com 10 dígitos e confirmou a mensagem esperada: “Informe um CPF com 11 dígitos ou um CNPJ com 14 dígitos.” Os demais casos foram associados a testes automatizados identificáveis na suíte. O relatório do Testiny registra precondições, passos, resultados esperados, comentários de evidência, autor e data de cada resultado.

4.3 Avaliação

O critério funcional foi atendido: 10 de 10 casos aprovados, nenhuma falha, nenhum bloqueio e nenhum caso não executado. A conclusão é limitada às regras representadas pelos casos. Fluxos de inteligência artificial, administração, recuperação de falhas, segurança ofensiva e compatibilidade entre navegadores não fizeram parte desta execução.

5 Relato do teste de caixa-branca

5.1 Objetivo e procedimento

O teste de caixa-branca mediu quais partes da implementação do aplicativo `financeiro` foram exercitadas pela suíte. O comando de cobertura executou os 29 testes com rastreamento de decisões. Migrações e o próprio arquivo de testes foram excluídos do denominador, pois o objetivo era avaliar o código de produção.

Foram considerados três indicadores: cobertura de instruções, correspondente às linhas executáveis visitadas; cobertura de decisões, correspondente aos destinos de ramificação percorridos; e cobertura combinada, calculada sobre as oportunidades de execução de instruções e decisões.

5.2 Resultados de cobertura

Indicador	Coberto	Total	Percentual
Instruções	831	954	87,1%
Decisões	88	138	63,8%
Cobertura combinada	919	1.092	84,2%

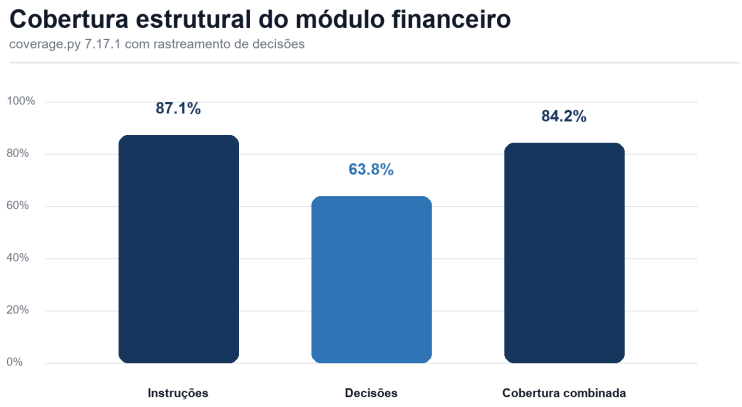


Figura 1 - Cobertura estrutural do aplicativo financeiro.

Os arquivos centrais apresentaram cobertura elevada: `views.py` alcançou 95,8%, `forms.py` 95,0%, `filters.py` 94,7% e `services.py` 87,9%. `models.py`, que concentra regras e propriedades de domínio, atingiu 84,0%. Os pontos de menor cobertura foram `pagination_tags.py`, com 38,9%, e o comando `popular_financeiro.py`, com 0%, porque a carga de dados demonstrativos não é chamada pela suíte funcional.

5.3 Interpretação

A cobertura de instruções indica que a maior parte do código principal foi exercitada. A diferença de 23,3 pontos percentuais entre instruções e decisões mostra que muitos condicionais foram visitados por apenas um resultado lógico. Os próximos testes estruturais devem priorizar ramificações alternativas em `models.py`, filtros, formulários e serviços. O comando de população deve receber testes próprios apenas se for tratado como parte entregável do produto; sua ausência reduz a métrica global, mas não invalida os casos funcionais avaliados.

Cobertura não equivale a correção. Um teste pode executar uma linha sem verificar adequadamente seu efeito. Por isso, a medida foi interpretada junto com as asserções funcionais e não como critério único de qualidade.

6 Análise de acessibilidade

6.1 Objetivo e escopo

A análise avaliou seis telas: página inicial, autenticação, painel financeiro, lista de pessoas, cadastro de lançamento e lista de parcelas. O axe-core foi configurado para regras WCAG 2.0, 2.1 e 2.2 aplicáveis aos níveis A e AA. Também foram coletados idioma da página, hierarquia de títulos, pontos de referência, imagens sem texto alternativo, campos sem rótulo e uma sequência inicial de 15 focos por teclado no painel.

Todas as telas declararam `lang="pt-br"`, apresentaram um título de nível 1, continham regiões `nav`, `main` e `footer`, não possuíam imagens sem atributo `alt` e não apresentaram campos de formulário sem rótulo identificável. A amostra de teclado percorreu marca, início, painel, lançamentos, cadastros, botão de novo item, menu do usuário e cartões de conteúdo em sequência utilizável.

6.2 Achados

Cinco telas não apresentaram violações automáticas. A lista de parcelas apresentou uma violação séria da regra `label-content-name-mismatch`, que exige que o nome acessível contenha o texto visível do componente. Foram afetados três atalhos de indicadores: parcelas pagas, parcelas em aberto e parcelas vencidas não pagas.

Tela	Regra violada	Impacto	Elementos afetados
Lista de parcelas	`label-content-name-mismatch`	Sério	3

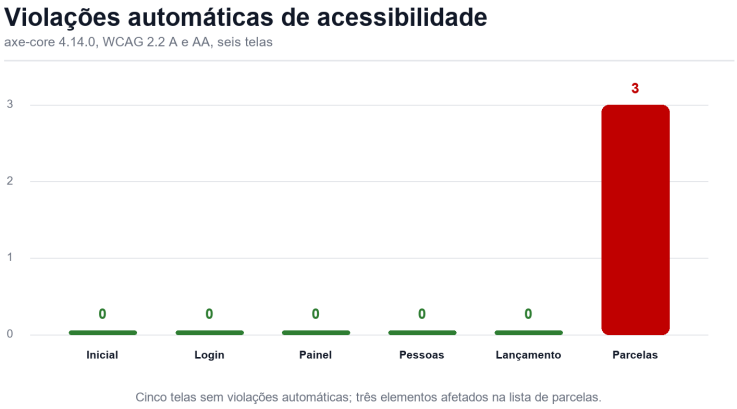


Figura 2 - Elementos afetados por violações automáticas de acessibilidade.

O relatório também classificou como inconclusivos quatro botões de avatar, porque o conteúdo visível é a inicial “A” e o nome acessível é “Menu do usuário admin”. Cinco campos `select` do cadastro de lançamento ficaram pendentes de revisão manual de contraste, pois a ferramenta não determinou a cor de fundo devido à imagem aplicada pelo componente. Esses itens não foram contados como violações confirmadas, mas exigem verificação humana.

6.3 Recomendações

Os três atalhos devem receber nomes acessíveis que preservem o texto visível e acrescentem contexto sem substituí-lo. O contraste dos campos de seleção deve ser medido nos estados normal, foco, desabilitado e erro. A avaliação deve ser complementada com leitor de tela, ampliação a 200%, refluxo em 320 CSS pixels, operação completa sem mouse e inspeção de mensagens dinâmicas. A automação detecta somente parte dos problemas possíveis e não sustenta, sozinha, uma declaração de conformidade integral com a WCAG.

7 Teste de carga

7.1 Objetivo e cenário

O ensaio observou a resposta da aplicação a consultas simultâneas em sessão autenticada. Cada usuário virtual percorreu repetidamente o painel financeiro, a lista de lançamentos, a lista de pessoas e a lista de parcelas. Foram executados três estágios de 15 segundos, com 10, 25 e 50 usuários virtuais. A sessão foi autenticada antes do início de cada estágio para evitar que a gravação de sessões dominasse a medição.

O ambiente usou o servidor de desenvolvimento do Django e SQLite local. Portanto, o ensaio é diagnóstico e não representa a capacidade da implantação real. Ele permite comparar tendências sob condições constantes, mas não definir um acordo de nível de serviço.

7.2 Resultados

Usuários virtuais	Requisições	Erros	Vazão	Mediana	p95	p99
10	246	0	16,09 req/s	600 ms	982 ms	1.221 ms
25	235	0	15,07 req/s	1.482 ms	3.042 ms	3.587 ms
50	250	0	15,54 req/s	3.021 ms	5.562 ms	7.596 ms

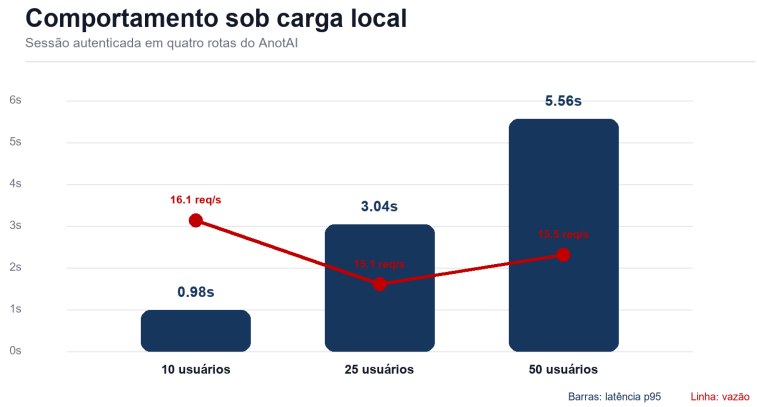


Figura 3 - Tendência de latência p95 e vazão no ensaio de carga local.

Ao todo, 731 requisições retornaram HTTP 200, sem erro ou redirecionamento indevido. A lista de parcelas foi a rota mais lenta: seu p95 passou de 1.192 ms com 10 usuários para 7.126 ms com 50 usuários. A vazão global não aumentou de forma relevante quando a concorrência subiu, enquanto a latência cresceu de maneira acentuada. Esse padrão indica saturação do ambiente local próximo de 15 a 16 requisições por segundo.

7.3 Avaliação e próximos ensaios

O sistema permaneceu funcional nos três estágios, mas o tempo de resposta com 25 e 50 usuários ultrapassou um limite prático de dois segundos para a maior parte das interações. Antes de atribuir o resultado ao código da aplicação, é necessário repetir o teste com servidor WSGI de produção, PostgreSQL, coleta de CPU e memória, aquecimento controlado e massa de dados semelhante à real. A lista de parcelas deve ser investigada primeiro, com análise de consultas, paginação e índices.

8 Síntese dos resultados e riscos

Dimensão	Evidência	Situação	Ação recomendada
Caixa-preta	10 de 10 casos aprovados no Testiny; 29 testes Django sem falhas	Atendido no escopo	Manter a suíte como regressão
Caixa-branca	87,1% de instruções e 63,8% de decisões	Parcialmente atendido	Criar casos para decisões não percorridas
Acessibilidade	1 regra violada em 3 elementos; itens inconclusivos	Requer correção e teste manual	Ajustar nomes acessíveis e validar contraste
Carga	731 respostas HTTP 200; p95 de 0,98 s a 5,56 s	Estável, porém com saturação local	Repetir em ambiente semelhante ao de produção

Os principais riscos para a interpretação são a diferença entre o ambiente local e a produção, a cobertura limitada a seis telas na auditoria de acessibilidade, o curto tempo de cada estágio de carga e a ausência de testes com tecnologias assistivas. Também não foi executada revisão de segurança ofensiva. Esses limites não anulam os resultados; definem até onde as conclusões podem ser aplicadas.

9 Conclusão

O módulo financeiro atendeu aos resultados funcionais planejados: todos os casos de caixa-preta e todos os testes automatizados executados foram aprovados. A análise de caixa-branca mostrou boa cobertura de instruções e revelou uma oportunidade clara de ampliar a cobertura de decisões. A acessibilidade apresentou estrutura básica consistente, mas os três atalhos da lista de parcelas precisam de nomes acessíveis compatíveis com o texto visível, além de revisão manual de contraste e interação. O teste de carga não encontrou erros, porém mostrou crescimento expressivo da latência e estabilização da vazão, o que justifica um novo ensaio em infraestrutura de produção.

O conjunto de evidências permite encerrar esta rodada de testes com aprovação funcional e pendências não funcionais registradas. A próxima iteração deve corrigir os achados de acessibilidade, cobrir ramificações não exercitadas e repetir a medição de desempenho com instrumentação e ambiente representativos.

Referências

1. DEQUE SYSTEMS. **axe-core: Accessibility engine for automated Web UI testing**. Disponível em: https://github.com/dequelabs/axe-core. Acesso em: 3 nov. 2026.
2. DJANGO SOFTWARE FOUNDATION. **Writing and running tests**. Disponível em: https://docs.djangoproject.com/en/5.2/topics/testing/overview/. Acesso em: 3 nov. 2026.
3. FOCUS. **Documento de Testes**. Centro de Informática da UFPE, 2010. Disponível em: https://www.cin.ufpe.br/~focus/documentos/gerencia/Documento_de_Testes.pdf. Acesso em: 3 nov. 2026.
4. ISO. **ISO/IEC/IEEE 29119-1:2022 Software and systems engineering - Software testing - Part 1: General concepts**. 2022. Disponível em: https://www.iso.org/standard/81291.html. Acesso em: 3 nov. 2026.
5. ISTQB. **ISTQB Glossary: black-box testing**. Disponível em: https://glossary.istqb.org/en_US/term/black-box-testing/2. Acesso em: 3 nov. 2026.
6. PRUDENCIO, I. **QA Quality Analytics e documentação de testes**. Medium, 2024. Disponível em: https://medium.com/@icaroprudencio/qa-quality-analytics-documenta%C3%A7%C3%A3o-de-testes-d12bac029064. Acesso em: 3 nov. 2026.
7. TESTINY. **Test management, test plans, test runs and reporting**. Disponível em: https://www.testiny.io/. Acesso em: 3 nov. 2026.
8. W3C. **Web Content Accessibility Guidelines WCAG 2.2**. Recomendação de 12 dez. 2024. Disponível em: https://www.w3.org/TR/WCAG22/. Acesso em: 3 nov. 2026.
9. COVERAGE.PY. **Branch coverage measurement**. Disponível em: https://coverage.readthedocs.io/en/latest/branch.html. Acesso em: 3 nov. 2026.

Apêndice A Evidências produzidas

- Plano de Testes formal em DOCX e PDF.
- Projeto ANOT no Testiny com 10 casos funcionais.
- Plano de Testes no Testiny associado aos casos.
- Execução TR-1 fechada com 100% de conclusão e 10 resultados aprovados.
- Relatório PDF da execução gerado pelo Testiny.
- Relatório JSON de cobertura com decisões.
- Relatório JSON da análise axe-core.
- Relatório JSON do teste de carga local.